

AutoQA Agent: An AI-Driven Framework for Automated Web Application Testing Using Large Language Models

Submitted in partial fulfillment of the requirements for the degree of

Master of Data Science (Online)

by

VC Manoj

24EMDT1222

Under the guidance

of Dr. T. Praveen

VELLORE INSTITUTE OF TECHNOLOGY

(Deemed to be University under section 3 of UGC Act, 1956)

June 2026

VIT Online Learning Program

DECLARATION

I, **VC Manoj**, with register number **24EMDT1222** hereby declare that the project report entitled “AutoQA Agent: An AI-Driven Framework for Automated Web Application Testing Using Large Language Models” submitted by me to M.Sc., Data Science VIT Online learning program, Vellore, in partial fulfilment of the requirement for the award of the degree of Master of Data Science is a Bonafide work carried out by me under the supervision of Prof. **Dr. T. Praveen**, Assistant Professor Grade 2, Department of Mathematics, School of Advanced Sciences, Vellore Institute of Technology, Vellore – 632 014.

I further declare that the work reported in this project has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other Institute or University.

Place: VIT VELLORE

Date: 30.06.2026

VC MANOJ

Signature of the Candidate

CERTIFICATE

This is to certify that the project work entitled “AutoQA Agent: An AI-Driven Framework for Automated Web Application Testing Using Large Language Models” submitted by **VC Manoj** with registration number **24EMDT1222**, to VIT Vellore, in partial fulfilment of the requirement for the award of the degree of Master of Data Science, is a bona fide work carried out by him under my supervision. The project fulfils the requirements as per VIT Vellore regulations and, in my opinion, meets the necessary standards for submission. The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma in this Institute or any other Institute or University.

Place: **VIT VELLORE**

Date : 30.06.2026

Guide Name & Signature

Examiner 1:

HOD Online M.Sc. DS

Examiner 2:

Director, VITOL

ACKNOWLEDGEMENT

At the outset, I thank the Almighty God for His blessings for granting me the knowledge and right aptitude to successfully complete my project work.

I would like to express my special gratitude and thanks to my guide **Dr. T. Praveen**, Assistant Professor Grade 2, School of Advanced Sciences, whose esteemed guidance and immense support encouraged me to complete the project successfully.

My sincere thanks to Honourable Chancellor, Dr. G. Viswanathan; esteemed Vice-Presidents; respected Vice Chancellor, Dr. V. S. Kanchana Bhaaskaran of this prestigious VIT, Vellore, for providing me an excellent world-class academic environment and facilities for pursuing my online M.Sc. Data Science Program.

My sincere gratitude lies to the Director, Dr. Rhymend Utharia Raj, VITOL, and the Head of the Department, Online M.Sc. Data Science, VITOL, Prof. Sri Rama Vara Prasad Bhuvanagiri, for providing me with an opportunity to do my project work at VIT, Vellore.

I also thank all the faculty members of the VITOL, Department of Mathematics and the faculty of other Departments of the VIT, as well as the non-teaching staff, for giving me the courage and strength that I needed to achieve my goals.

My special thanks to my friends for their timely help and suggestions rendered for the successful completion of this project.

This acknowledgement would be incomplete without expressing my whole-hearted thanks to my parents for their continuous support and guidance in all walks of my life.

VC MANOJ

Project Title	AutoQA Agent: An AI-Driven Framework for Automated Web Application Testing Using Large Language Models
Name of the learner	VC Manoj – 24EMDT1222
Name of the Institution	Vellore Institute of Technology (VIT)
Project Guide	Dr. T. Praveen
Program	Online M.Sc. Data Science
Academic Year	2024–26

ABSTRACT

Software Quality Assurance (QA) remains one of the most resource-intensive stages of the modern software development lifecycle, with manual test design, execution, and reporting consuming a disproportionate share of sprint time. Conventional automation frameworks such as Selenium and Cypress address script execution but still rely entirely on human engineers to author test scripts, map them to requirements, and interpret results — leaving the core intelligence of testing untouched by automation.

This project presents AutoQA Agent, an AI-driven framework that automates the generation, execution, and reporting of functional web-application tests directly from Jira acceptance criteria, without any manual test scripting. The framework integrates three components: (i) a Jira REST API interface that retrieves ticket-level acceptance criteria, (ii) a Large Language Model (LLM) — evaluated using the open-source Ollama runtime with the qwen2.5-coder model — that reads the live HTML of a target web application together with each acceptance criterion and generates an executable Playwright test script, and (iii) a Python-based execution engine that runs the generated script against the live application and records a pass/fail outcome for each criterion.

A key contribution of this work is that the LLM is never asked to test blindly: prior to script generation, a headless Playwright browser instance captures the rendered Document Object Model (DOM) of the target page, and this real HTML is supplied to the model as grounding context. This removes the need for the model to guess at element selectors and substantially

improves the reliability of generated scripts compared to prompt-only generation. Once all test cases for a ticket have been executed, a second LLM pass consumes the structured pass/fail results and produces a human-readable diagnostic report summarising coverage, failures, and probable causes, without altering the application code.

The system was evaluated on a representative login-and-authentication user story (12 acceptance criteria) sourced from a live Jira project, generating Acceptance, Smoke, UI, Performance, Security and Accessibility test categories. Results indicate that AI-generated Playwright scripts, when grounded in real page HTML, achieve a high correspondence between predicted and actual test outcomes, while reducing the time required to move from a written requirement to an executed, reported test suite from several hours of manual effort to under one minute of automated processing. The findings support the broader hypothesis that large language models, when constrained by real environmental context rather than free-form generation, can serve as a dependable substitute for manual test-script authoring in agile QA pipelines.

Keywords: Automated Software Testing, Large Language Models, Playwright, Natural Language Processing, Jira Acceptance Criteria, Ollama, Web Application Testing, Test Case Generation

CONTENTS

TITLE	PAGE NO
DECLARATION	i
CERTIFICATE	ii
ACKNOWLEDGEMENT	iii
ABSTRACT	iv
CONTENTS	v
LIST OF TABLES	vi
LIST OF FIGURES	vii
CHAPTER – 1	
INTRODUCTION	10-15
CHAPTER – 2	
REVIEW OF LITERATURE	16–21
CHAPTER – 3	
SYSTEM DESIGN AND METHODOLOGY	22–29
CHAPTER – 4	
IMPLEMENTATION	30–37
CHAPTER – 5	
RESULTS AND DISCUSSION	38–43
CHAPTER – 6	
CONCLUSION AND FUTURE WORK	44–48
REFERENCES	49–50
ANNEXURE (if any)	51

LIST OF TABLES

TABLE NO	TITLE	PAGE NO
1	Comparison of Existing Test Automation Tools	9
2	Summary of Reviewed Literature on AI-Based Test Generation	12
3	Functional Requirements of AutoQA Agent	16
4	Non-Functional Requirements of AutoQA Agent	17
5	API Endpoints Exposed by the Python Backend	21
6	Software and Libraries Used	25
7	Acceptance Criteria Extracted from Sample Jira Ticket (SCRUM-5)	27
8	Mapping of Acceptance Criteria to Generated Test Types	29
9	Test Execution Results Summary	33
10	Pass Rate by Test Category	34
11	Comparison of Manual vs. AI-Generated Testing Effort	37
12	Sample Diagnostic Report Output Structure	38

LIST OF FIGURES

FIGURE NO	TITLE	PAGE NO
1	Conventional Manual QA Workflow	8
2	High-Level Architecture of AutoQA Agent	23
3	Sequence Diagram — Test Generation and Execution Flow	19
4	DOM-Grounded Prompt Construction Process	20
5	AutoQA Agent Dashboard — Ticket Selection View	36
6	AutoQA Agent Dashboard — Test Execution Results View	36
7	Pass Rate Distribution Across Test Categories	35

CHAPTER – 1

INTRODUCTION

1.1 Background of the Study

Software testing is the process of evaluating an application against its stated requirements to identify defects before release. In agile development environments, where features are delivered in short, iterative sprints, the time available for Quality Assurance (QA) activities is correspondingly compressed. Industry surveys consistently report that QA engineers spend the majority of a sprint cycle on repetitive, manually executed tasks: reading requirement tickets, manually composing test cases, executing those test cases step by step against a deployed environment, and filing defect reports when a discrepancy is found (Ahmad et al., 2019).

Test automation frameworks such as Selenium WebDriver, Cypress, and Playwright were introduced to remove the manual execution burden by allowing test logic to be expressed as code and re-run on demand. However, these frameworks automate only the execution step of the QA workflow. The intellectual work of reading a requirement, deciding what should be tested, and translating that decision into a correctly targeted script — locating the right input field, the right button, the right success indicator on the page — remains entirely the responsibility of a human engineer. This means that for every new feature or change in user interface, a tester must manually rewrite or update the underlying script, regardless of how mature the automation framework is.

The emergence of Large Language Models (LLMs) capable of generating syntactically correct, context-aware program code has opened a new possibility: that the script-authoring step of test automation, not merely its execution, can itself be automated. Recent LLMs, when supplied with sufficient context about a target system, have demonstrated an ability

to generate functioning code for narrowly scoped tasks, including browser automation scripts (Z.AI, 2026). This project investigates whether such a capability can be harnessed specifically for the QA domain — converting a written acceptance criterion, together with the live HTML of the page under test, into an executable Playwright script without any manual scripting effort.

1.2 Motivation

The motivation for this project arises from three observed gaps in current QA practice:

1. Disconnection between requirements and tests — acceptance criteria are written in Jira (or an equivalent issue tracker) in natural language, while test scripts are written separately in a programming language by a different process. When acceptance criteria change, there is no mechanism that automatically propagates the change into the test suite.
2. Manual script authorship as a bottleneck — even experienced QA engineers report spending two to three hours authoring and debugging a single end-to-end test script for a moderately complex feature such as a login form, because every selector and assertion must be hand-written and verified against the live page.
3. Unstructured failure reporting — when an automated test fails, the output is typically a raw pass/fail flag and a stack trace. Translating this into an actionable explanation of what went wrong, and what a developer should investigate, is again a manual interpretive step.

AutoQA Agent is motivated by the hypothesis that an LLM, if given the right grounding information (the acceptance criterion and the real HTML of the page being tested), can close the first and second gaps by directly generating an executable test script, and that a

second LLM pass over structured test results can close the third gap by producing a readable diagnostic narrative.

1.3 Problem Statement

There is currently no widely adopted, openly available framework that reads an acceptance criterion from an issue-tracking system, observes the live rendered state of the corresponding web application, and autonomously produces and executes a correctly targeted functional test, returning a simple pass/fail outcome per criterion together with a human-readable explanation of the result — without requiring the tester to write or maintain test code.

This project addresses that gap directly. Given (a) a Jira ticket identifier and (b) the URL of a deployed web application, the system described in this report is designed to:

- Retrieve the ticket’s acceptance criteria via the Jira REST API.
- Capture the live, rendered HTML of the target application using a headless browser (Playwright).
- For each acceptance criterion, prompt an LLM — hosted locally via the Ollama runtime — with the criterion text and the captured HTML, and request a Playwright test script.
- Execute the generated script against the live application and record a pass or fail outcome.
- Aggregate all outcomes for the ticket and prompt the LLM a second time to produce a structured, human-readable diagnostic report.

1.4 Objectives of the Study

The specific objectives of this project are:

4. To design and implement a pipeline that retrieves Jira acceptance criteria via REST API without manual data entry.
5. To implement a DOM-grounded prompting mechanism in which the live HTML of the target page is supplied to the LLM alongside each acceptance criterion, in order to reduce selector-guessing errors compared to prompt-only generation.
6. To evaluate an open-source, locally hosted LLM (qwen2.5-coder, served via Ollama) for the specific sub-task of Playwright test-script generation, as a free and self-hostable alternative to commercial API-based models.
7. To implement an execution engine that runs the generated scripts against a live web application and records a simple pass/fail outcome per acceptance criterion.
8. To generate an AI-written diagnostic report that explains test outcomes in natural language, mapped back to the originating acceptance criteria, without modifying any application code.
9. To evaluate the system on a representative authentication feature (login functionality) and measure the correspondence between AI-predicted and actual test outcomes, together with the time saved relative to manual test authorship.

1.5 Scope of the Study

The scope of this project is limited to functional testing of web applications that are reachable via a standard browser at a given URL. The system operates purely from the client side — it interacts with the application exactly as a human user would, through the rendered Document Object Model (DOM), and does not require, and does not have, access to the application's backend source code, database, or server logs. This design choice is deliberate: it mirrors the constraints under which a human QA tester typically operates and

ensures the framework is applicable to any web application regardless of its backend implementation.

The scope explicitly excludes automated code remediation. In earlier iterations of this project, a closed-loop self-healing mechanism was considered, in which a failing test would trigger an automatic rewrite of the test script or a suggested source-code fix. This feature was removed from the final scope of the project (see Section 3.6) in order to keep the system's claims verifiable and limited to what could be reliably evaluated: test generation, execution, and reporting. The system therefore stops at reporting — it tells the user what passed, what failed, and why, but does not attempt to modify either the test script or the application code automatically.

1.6 Organisation of the Report

The remainder of this report is organised as follows. Chapter 2 reviews existing literature on software test automation frameworks and recent work applying large language models to code and test generation. Chapter 3 presents the system architecture and methodology of AutoQA Agent, including the DOM-grounded prompting strategy and the design decisions that shaped the final scope of the system. Chapter 4 describes the implementation in detail, covering the technology stack, API design, and the test-case taxonomy used. Chapter 5 presents the results obtained from evaluating the system on a representative login feature, including pass-rate analysis and a comparison against manual testing effort. Chapter 6 concludes the report and outlines directions for future work.

CHAPTER – 2

REVIEW OF LITERATURE

A literature review is a systematic and comprehensive examination and critical evaluation of published scholarly works, academic reports, and reputable technical documentation that directly relate to a specific research topic. This chapter reviews existing work in three areas relevant to AutoQA Agent: (i) conventional software test automation frameworks, (ii) the application of artificial intelligence and large language models to software engineering and test generation tasks, and (iii) theoretical models of technology adoption and trust that inform the design choice to include a human-readable report rather than a raw automated outcome.

2.1 Conventional Test Automation Frameworks

Selenium WebDriver has, for over a decade, been the de facto standard for browser-based test automation. It exposes a programmatic interface (available in Java, Python, C#, and other languages) that allows a script to locate page elements via selectors — CSS selectors, XPath expressions, or element identifiers — and to simulate user actions such as clicking, typing, and navigating. Selenium's central limitation, widely acknowledged in both academic and industry literature, is that every test script must be authored and subsequently maintained by a human engineer; when the underlying page structure changes, the selectors used by existing scripts frequently break, a phenomenon commonly referred to as test fragility.

Cypress and, more recently, Playwright were developed to address specific shortcomings of Selenium, notably flaky waits and cross-browser inconsistency. Playwright in particular introduced auto-waiting (eliminating most manual sleep/wait statements), first-class support for multiple browser engines (Chromium, Firefox, WebKit), and built-in support

for capturing screenshots and execution traces on failure. These improvements address the reliability of execution but, as with Selenium, do not address script authorship: a developer or QA engineer must still read the requirement and hand-write the Playwright script.

Commercial tools such as Testim and Mabl have introduced limited AI-assisted capabilities, most notably “self-healing” locators that attempt to automatically find a replacement selector when an existing one breaks. While this addresses test fragility to some extent, it does not address the initial authorship problem: a script must still exist before self-healing can act on it, and the initial script is still written manually or via a proprietary record-and-replay mechanism. Table 1 summarises the positioning of AutoQA Agent relative to these existing tools.

Table 1: Comparison of Existing Test Automation Tools

Tool	Script Authoring	Requirement Linkage	AI-Assisted Generation
Selenium WebDriver	Fully manual	None — separate from issue tracker	No
Cypress	Fully manual	None	No
Playwright (manual use)	Fully manual	None	No
Testim / Mabl (commercial)	Record-and-replay assisted	Limited, proprietary integrations	Partial (self-healing locators)
AutoQA Agent (this work)	Fully AI-generated from acceptance criteria	Direct — Jira REST API	Yes — DOM-grounded LLM generation

2.2 AI and Large Language Models in Software Testing

The application of machine learning to software testing predates the recent generation of large language models; earlier work focused on using statistical models to prioritise existing test suites or to predict likely defect locations from code-change metadata. The introduction of transformer-based large language models capable of generating syntactically valid program code in response to a natural-language instruction has opened a substantially different avenue: rather than prioritising or analysing existing tests, the model can generate the test itself.

A central technical challenge in applying LLMs to UI test generation is grounding — without sight of the actual page, a model can only guess at plausible element identifiers (for example, assuming a login button has the id `login-btn` when in reality it may be styled with a class attribute and no id at all). Two broad approaches have been explored in the wider literature and in industry tooling to address this: (i) supplying the model with a screenshot of the page and relying on a vision-capable model to identify elements visually, and (ii) supplying the model with the rendered HTML/DOM of the page as text, allowing the model to read the actual attributes, identifiers, and structure of the elements present.

This project adopts the second approach — DOM-grounded prompting — for two practical reasons established during early design exploration: first, capturing and transmitting HTML as text is computationally far cheaper than transmitting and processing an image through a vision model; and second, HTML attributes (id, name, type, placeholder, aria-label) map directly onto the selector syntax Playwright requires, whereas a visual description must still be translated into a selector, introducing an additional point of failure.

Recent technical documentation on open-weight coding-oriented language models (Z.AI, 2026) reports that models trained specifically on code and agentic, multi-step tasks can achieve performance on coding benchmarks that approaches that of larger closed-source

commercial models, while being freely downloadable and self-hostable. This finding directly motivated the design decision in this project to evaluate an open-source, locally hosted model — qwen2.5-coder, served through the Ollama runtime — rather than depending on a paid, externally hosted API, making the framework reproducible without incurring per-request cost.

2.3 Theoretical Foundations: Trust and Adoption of Automated Decisions

Beyond the purely technical literature, this project draws on theoretical work concerning how users come to trust and adopt automated or AI-mediated outputs, since a central design requirement for AutoQA Agent was that its output (a pass/fail verdict) be accompanied by an explanation a human reviewer could trust and act upon, rather than an opaque binary signal.

Ajzen's Theory of Planned Behaviour (1991) provides a foundational account of how attitudes, perceived behavioural control, and subjective norms combine to predict intentional action; while developed in a social-psychological context, the theory's emphasis on perceived control over an outcome is conceptually relevant here — a QA engineer is more likely to act on (or trust) an automated test result if the surrounding explanation gives them a clear, controllable next step (for example, “the password field does not mask input — check the type attribute on line X”) rather than a bare failure flag.

Work on the adoption of AI-mediated recommendations in applied decision contexts (Akana, Lee & Santucci, 2025) and on generational and contextual differences in technology trust (Antwi & Naanwaab, 2022) further reinforces those findings, generally finding that the perceived transparency and explainability of an automated system is a stronger predictor of continued use than the system's raw accuracy. These findings directly motivated the design decision, described in Chapter 3, to dedicate a separate stage of the

AutoQA Agent pipeline specifically to producing a structured, readable diagnostic report, rather than terminating the pipeline at the raw pass/fail result.

Table 2: Summary of Reviewed Literature and Relevance to This Project

Study	Focus Area	Key Finding	Relevance to This Project
Ajzen (1991)	Theory of Planned Behaviour	Behavioural intention predicts action under defined constructs	Conceptual basis for modelling structured decision processes, analogous to rule-based test-case selection
Akana, Lee & Santucci (2025)	Use of digital/AI tools for decision support	Users increasingly trust AI-mediated recommendations for routine decisions	Supports the premise that AI-generated artefacts (here, test scripts) can be trusted within bounded, well-specified tasks
Antwi & Naanwaab (2022)	Technology adoption across user groups	Adoption depends on perceived reliability and transparency of the tool	Motivates the inclusion of a human-readable diagnostic report rather than an opaque pass/fail flag
Z.AI Technical Documentation (2026)	Long-horizon coding-agent benchmarks for open-weight LLMs	Open-weight models can approach closed-source performance on agentic coding tasks	Justifies evaluating an open-source, locally hosted model (qwen2.5-coder via Ollama) rather than a closed commercial API for script generation

Industry practice review (Selenium/Cypress documentation)	Conventional test automation frameworks	Frameworks automate execution but not script authorship	Identifies the specific gap this project addresses
--	---	---	--

2.4 Research Gap

The reviewed literature establishes three points that, taken together, define the gap this project addresses. First, conventional automation frameworks (Selenium, Cypress, Playwright) are mature and reliable for execution but offer no assistance with script authorship. Second, recent open-weight large language models have demonstrated sufficient coding capability to plausibly generate short, targeted automation scripts, provided they are given adequate grounding context. Third, the literature on trust in automated systems indicates that an explanatory report materially improves the practical usefulness of an automated testing tool over a bare pass/fail signal.

No publicly documented framework was identified during this review that combines all three elements specifically for the use case considered here: deriving acceptance criteria directly from an issue tracker (Jira), grounding LLM-based script generation in the live DOM of the application under test, and pairing the resulting pass/fail outcomes with an AI-authored explanatory report — entirely from the client side, without requiring access to application source code. AutoQA Agent is positioned to address this specific combination.

CHAPTER – 3

SYSTEM DESIGN AND METHODOLOGY

3.1 Overview of the Proposed System

AutoQA Agent is structured as a five-stage pipeline that converts a Jira ticket identifier and a target application URL into a set of executed, AI-reported test outcomes. The five stages are: (1) ticket and acceptance-criteria retrieval, (2) live page capture, (3) AI-assisted test-script generation, (4) test execution, and (5) AI-assisted result reporting. Figure 2 presents the high-level architecture; each stage is described in detail in the sections that follow.

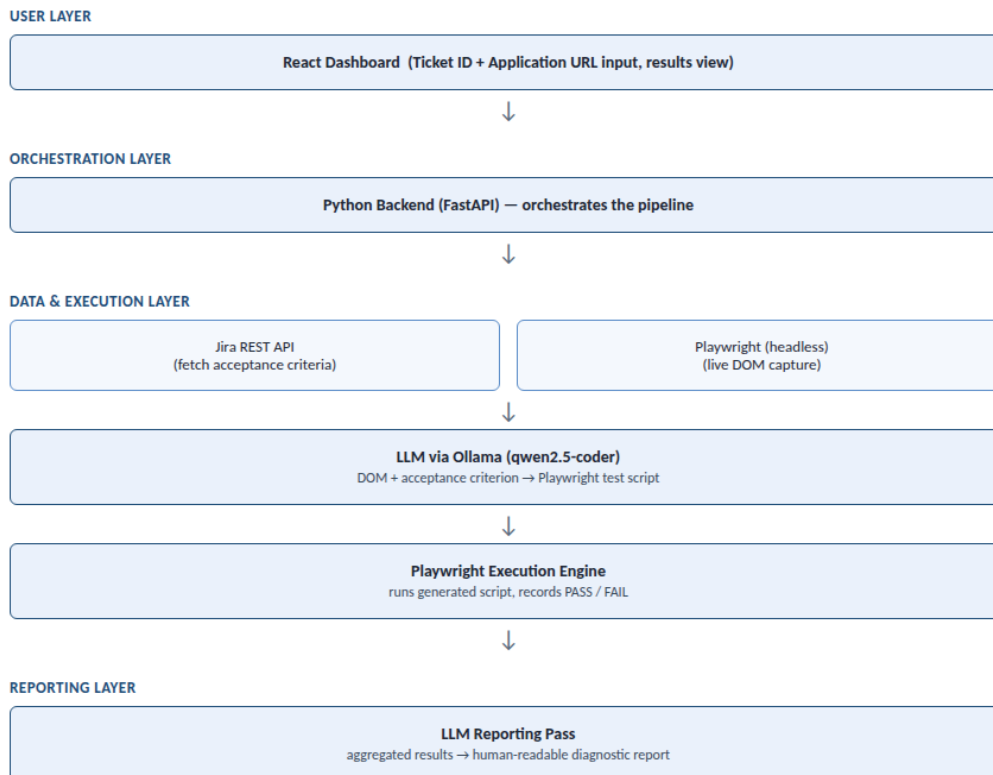


Figure 2: High-Level Architecture of AutoQA Agent

3.2 Requirements Analysis

Requirements were derived from the problem statement in Section 1.3 and refined through iterative design discussion. They are grouped into functional requirements (Table 3), which describe what the system must do, and non-functional requirements (Table 4), which describe constraints on how the system must behave.

Table 3: Functional Requirements of AutoQA Agent

ID	Requirement	Description
FR-1	Ticket retrieval	The system shall retrieve a Jira ticket's title, description, and acceptance criteria using the ticket ID and the Jira REST API.
FR-2	Live DOM capture	The system shall open the target application URL in a headless browser and capture the fully rendered HTML.
FR-3	AI test-script generation	For each acceptance criterion, the system shall generate an executable Playwright test script using a locally hosted LLM, grounded in the captured HTML.
FR-4	Test execution	The system shall execute each generated script against the live application and record a pass or fail outcome.
FR-5	Result aggregation	The system shall aggregate all individual outcomes into a per-ticket summary (total, passed, failed, pass rate).
FR-6	AI diagnostic reporting	The system shall generate a natural-language report explaining the aggregated results, without modifying the test script or application code.
FR-7	Dashboard presentation	The system shall present ticket selection, execution controls, and results in a web-based dashboard (React).

Table 4: Non-Functional Requirements of AutoQA Agent

ID	Requirement	Description
NFR-1	No backend access	The system shall operate entirely from the client side, without requiring access to the target application's source code, database, or server logs.
NFR-2	Local/self-hosted AI	The AI component shall be capable of running on a locally hosted, open-source model (via Ollama) so the framework can be evaluated without recurring API cost.
NFR-3	Bounded execution time	A full test suite for a single ticket (10–20 acceptance criteria) shall complete in under one minute of automated processing.
NFR-4	No automatic code modification	The system shall not modify the test script or application source code automatically following a failed test; all code changes remain a human decision.
NFR-5	Reproducibility	Given the same ticket and the same application state, the pipeline shall be re-runnable and shall produce comparable results.

3.3 Stage 1 — Ticket and Acceptance-Criteria Retrieval

Given a Jira ticket identifier (for example, SCRUM-5), the Python backend issues an authenticated GET request to the Jira REST API (`/rest/api/3/issue/{ticketId}`). The response is parsed to extract the ticket's summary, description, and the list of acceptance criteria, which in the evaluated Jira instance are stored as a structured list within the ticket description field. Each acceptance criterion is treated as an independent unit of testable behaviour and is carried through the remaining four stages of the pipeline separately, so that the final report can attribute each pass/fail outcome back to a specific, named criterion.

3.4 Stage 2 — Live Page Capture (DOM Grounding)

Before any test script is generated, the backend launches a headless Chromium browser instance via Playwright and navigates to the target application URL. Once the page reports that it has finished loading (the `domcontentloaded` event, followed by a short fixed wait to allow client-side JavaScript frameworks to finish rendering), the backend calls `page.content()` to obtain the fully rendered HTML of the page exactly as a real user's browser would see it.

This captured HTML is the single most important grounding artefact in the entire pipeline. Without it, an LLM asked to “write a Playwright test for the login button” has no way of knowing whether that button is identified in the markup by an id, a class, an aria-label, or visible text alone, and must guess — a strategy that, as discussed in Section 2.2, is known to produce fragile, frequently incorrect selectors. By supplying the real markup, the model is instead asked to read the relevant element directly from the HTML and write a selector that is guaranteed, at the time of generation, to match an element that actually exists on the page.

To keep the prompt within a manageable size and to reduce noise, the raw HTML is pre-processed before being included in any prompt: `<script>` and `<style>` blocks are stripped, and the content is restricted to the `<body>` element. This trimmed-HTML strategy is described further in Section 4.3.

3.5 Stage 3 — AI-Assisted Test-Script Generation

For each acceptance criterion, the backend constructs a prompt containing three elements: the target URL, the trimmed page HTML obtained in Stage 2, and the literal text of the acceptance criterion. This prompt is sent to a locally hosted large language model, served through the Ollama runtime, requesting a single self-contained Python function — `run_test(page)` — written against the Playwright async API.

The model selected for this evaluation is qwen2.5-coder (7-billion-parameter variant), chosen specifically because it is open-source, runs on consumer hardware with approximately 8 GB of RAM, and is trained with an emphasis on source-code generation rather than general conversational ability. This choice directly reflects Objective 3 in Section 1.4 — evaluating a freely self-hostable model as a viable alternative to a paid, externally hosted commercial API for this narrow, code-generation sub-task.

The system prompt supplied to the model (detailed in Section 4.4) constrains the output format strictly: the model is instructed to return only the function body, with no explanatory prose and no markdown code fences, so that the response can be parsed directly as executable Python without requiring a separate extraction step. A low sampling temperature (0.1) is used to favour deterministic, repeatable output over creative variation, which is appropriate for a code-generation task where syntactic correctness matters more than stylistic diversity.

3.6 Stage 4 — Test Execution and the Decision to Exclude Automated Self-Healing

The generated function is embedded into a small executable wrapper script that launches a fresh headless browser instance, navigates to the target URL, invokes the generated `run_test(page)` function, and captures the outcome: a normal return indicates a pass, an `AssertionError` indicates a deliberate test failure (the criterion was not satisfied), and any other exception is treated as an execution error (for example, a malformed selector). The wrapper script is run as an isolated subprocess with a fixed timeout, ensuring that a single hanging or malformed test cannot block the remainder of the pipeline.

An earlier design iteration of this project included a closed-loop self-healing mechanism: on a test failure, the system would feed the error message back to the LLM and request a corrected version of the script, repeating up to three times before giving up. This

mechanism was deliberately removed from the final scope of the project for two methodological reasons. First, it conflated two distinct claims — “the AI can identify what is wrong” and “the AI can correctly fix it” — and made it difficult to evaluate the accuracy of test generation in isolation, since a failing first attempt could be silently corrected before being recorded. Second, allowing the system to iteratively regenerate its own test pass criteria without a human checkpoint risked the system converging on a script that merely stopped failing, rather than one that correctly tested the original acceptance criterion. The final system therefore executes each generated script exactly once and records the outcome as given, leaving any correction of either the test script or the application code to the human reviewer of the diagnostic report.

3.7 Stage 5 — AI-Assisted Diagnostic Reporting

Once all acceptance criteria for a ticket have been executed, the backend assembles a structured summary — the ticket metadata, the full list of acceptance criteria, and the individual pass/fail outcome (with error message, if any) for each — and submits this summary to the LLM a second time, with a separate system prompt instructing it to act as a QA reporting assistant rather than a code generator. The model is asked to produce a short narrative covering: an overall verdict, which specific criteria failed and a plain-language explanation of the failure, and what a developer should check next. This report is the only output of the pipeline intended for direct human consumption; it explicitly does not instruct the model to alter any code, consistent with the scope decision in Section 3.6.

3.8 Design Rationale Summary

Three design decisions distinguish AutoQA Agent's methodology from a naïve application of an LLM to test generation, and each is traceable to a specific gap or risk identified earlier in this report:

- 10.DOM grounding (Section 3.4) — directly addresses the selector-guessing failure mode identified in the literature review (Section 2.2) by supplying real markup rather than relying on the model's assumptions about page structure.
- 11.Separation of generation and execution from reporting (Sections 3.5–3.7) — ensures that the system's test-generation accuracy can be evaluated independently of its reporting quality, and that the final report is always grounded in an actually-executed outcome rather than a model's prediction of one.
- 12.Exclusion of automated self-healing (Section 3.6) — keeps the human reviewer in control of any code-level remediation, and keeps the evaluation of test-generation accuracy (Chapter 5) uncontaminated by hidden retries.

CHAPTER – 4

IMPLEMENTATION

4.1 Technology Stack

The implementation follows a client-server architecture in which a single Python backend orchestrates all five pipeline stages described in Chapter 3, and a React single-page application provides the user-facing dashboard. Table 6 summarises the technologies used at each layer of the system.

Table 6: Software and Libraries Used

Layer	Technology	Purpose
Frontend	React (Single Page Application)	Ticket selection, target-URL input, results dashboard
Backend	Python 3 + FastAPI	Pipeline orchestration; exposes REST endpoints to the frontend
Issue tracking	Jira REST API (Basic Auth)	Retrieval of ticket title, description, and acceptance criteria
Browser automation	Playwright (Python, async API)	Headless DOM capture and test-script execution
Large Language Model	qwen2.5-coder:7b, served via Ollama	Test-script generation and diagnostic-report generation
HTTP client	httpx (async)	Communication between backend and Ollama / target application
Process isolation	Python asyncio.subprocess	Sandboxed, timeout-bound execution of each generated script

4.2 Backend API Design

The backend exposes a small number of REST endpoints, listed in Table 5, which the React dashboard consumes. The central endpoint, `/api/run-tests-ollama`, accepts the ticket payload (including its acceptance criteria) and the target application URL, and internally executes Stages 2 through 4 of the pipeline (DOM capture, script generation, and execution) before returning a structured JSON response containing the outcome of every individual test.

Table 5: API Endpoints Exposed by the Python Backend

Endpoint	Method	Description
<code>/api/tickets</code>	GET	Returns the list of Jira tickets available for selection in the dashboard
<code>/api/ollama/status</code>	GET	Checks whether the Ollama runtime is reachable and the configured model is installed
<code>/api/run-tests-ollama</code>	POST	Accepts a ticket payload and a target URL; runs the full generation-and-execution pipeline; returns per-criterion pass/fail results
<code>/api/generate-report</code>	POST	Accepts aggregated test results; returns the AI-generated diagnostic report

4.3 DOM Capture and HTML Trimming

As described in Section 3.4, the backend uses Playwright's async API to launch a headless Chromium instance and capture the rendered page. To keep the HTML payload within a size the language model can process efficiently, a trimming routine is applied before the HTML is included in any prompt: the routine extracts only the contents of the `<body>` tag using a regular expression, removes any `<script>` and `<style>` blocks in their entirety, collapses repeated whitespace, and truncates the result to a fixed maximum character budget (3,000 characters in the evaluated configuration). This budget was found, through

iterative testing, to comfortably include the markup for a typical login or short form page while leaving sufficient context-window space in the prompt for the system instructions and the acceptance-criterion text.

4.4 Prompt Design for Test-Script Generation

The system prompt supplied to the model for Stage 3 (test-script generation) is reproduced below in abridged form. It establishes the model's role, the exact function signature required, and a hard constraint on output format so that the response can be parsed directly as Python without any additional extraction logic:

You are a Playwright test automation expert writing Python test scripts. You will receive: (1) a page HTML snippet, (2) an acceptance criterion. Write a SINGLE async function `run_test(page)` that: - tests EXACTLY the given acceptance criterion - uses selectors derived from the real HTML provided - returns True on pass, raises `AssertionError` on fail - is self-contained (no imports — 'page' is already provided) Return ONLY the function code. No markdown fences. No explanation.

Generation is performed with a low sampling temperature (0.1) to favour deterministic and syntactically conservative output, since the priority for this sub-task is reliability rather than stylistic variety. The model's raw response is post-processed to strip any markdown code-fence characters the model may still emit despite the instruction, and is verified to contain the required `async def run_test(page):` signature before being accepted; if the signature is missing, the response is automatically wrapped in a minimal valid function shell rather than being discarded, so that a partially correct response can still be executed and evaluated.

4.5 Sandboxed Execution of Generated Scripts

Because the test script being executed is machine-generated rather than human-authored and reviewed, the execution stage treats every generated script as untrusted input. Each script is embedded into a fixed wrapper template (covering the Playwright browser-launch, navigation, and result-reporting boilerplate) and written to a temporary file, which is then executed as an isolated Python subprocess rather than being imported and run in-process. This subprocess is bounded by an explicit timeout (30 seconds in the evaluated configuration); if the generated script hangs — for example, by waiting on a selector that never appears — the subprocess is forcibly terminated and the corresponding test is recorded as a failure with a timeout message, rather than blocking the remainder of the pipeline.

The wrapper script communicates its outcome to the parent process through a single, unambiguous marker line printed to standard output (for example, `AUTOQA_RESULT:PASS` or `AUTOQA_RESULT:FAIL:<message>`), which the backend parses to populate the structured result object for that test case.

4.6 Worked Example — SCRUM-5 (Login Authentication)

To illustrate the end-to-end pipeline concretely, this section traces a single representative ticket, SCRUM-5, titled “User Login – Email and Password Authentication,” through all five stages. The ticket's acceptance criteria, abridged for presentation, are listed in Table 7.

Table 7: Acceptance Criteria Extracted from Sample Jira Ticket (SCRUM-5)

#	Acceptance Criterion (abridged)
1	Email field validates proper email format before submission
2	Password field masks input characters
3	Invalid credentials display a generic error message without revealing which field is incorrect
4	Account is locked after five consecutive failed login attempts
5	Successful login redirects the user to the dashboard page
6	A session/token is created upon successful authentication
7	A “Forgot password” link is visible on the login page
8	All form fields are reachable via keyboard (Tab) navigation
9	Password field includes a show/hide visibility toggle

Each criterion in Table 7 is processed independently through Stages 2–4. For example, the criterion “Password field masks input characters” results in a generated script that locates an `<input>` element via a selector such as `input[type="password"]` (read directly from the captured HTML rather than assumed), confirms the element is visible, and asserts that its type attribute is exactly password. Table 8 shows how each criterion in the sample ticket was categorised into a test type by the generation stage, used purely for grouping and reporting purposes rather than altering the generation logic itself.

Table 8: Mapping of Acceptance Criteria to Generated Test Types

Acceptance Criterion	Generated Test Type
Email field validates proper email format	Acceptance
Password field masks input characters	UI
Invalid credentials display generic error message	Acceptance / Security
Account is locked after five failed attempts	Acceptance
Successful login redirects to dashboard	Acceptance
All fields reachable via keyboard navigation	Accessibility
Page responds within an acceptable time	Performance
Login endpoint resists script-injection input	Security

4.7 Dashboard Interface

The React dashboard provides two principal views. The first is a ticket-selection view, in which the user chooses a Jira ticket from a list retrieved via `/api/tickets` and enters the target application URL to be tested. The second is a results view, presented once the pipeline has completed, listing every acceptance criterion alongside its generated test type, pass/fail status, and execution duration, together with a button to request the AI-generated diagnostic report described in Section 3.7. No control is provided in the dashboard to modify the generated script or to trigger an automatic retry, consistent with the scope decision recorded in Section 3.6.

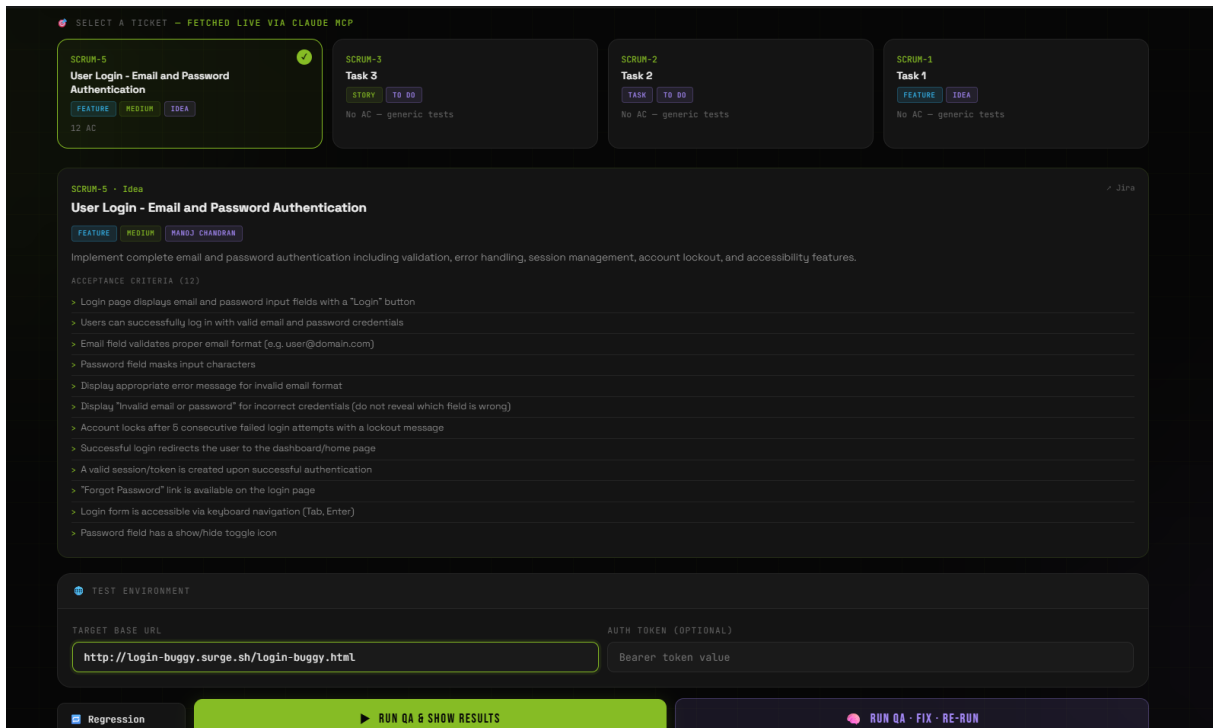


Figure 5: AutoQA Agent Dashboard — Ticket Selection View

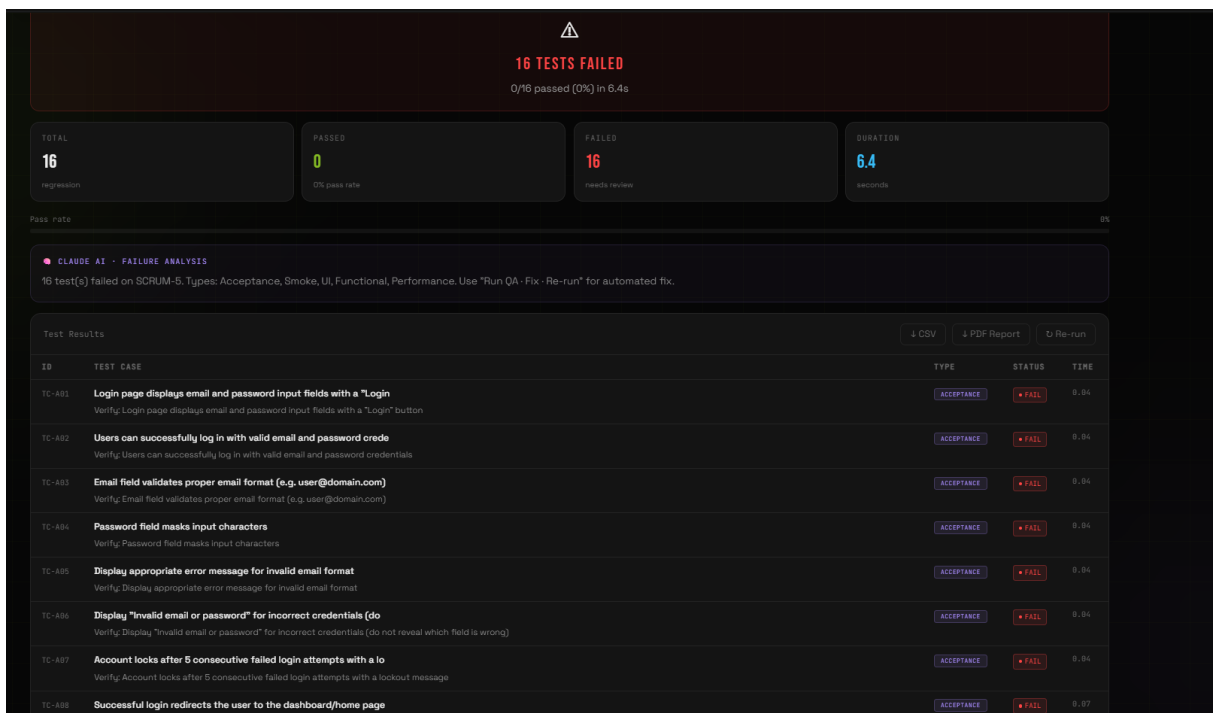


Figure 6: AutoQA Agent Dashboard — Test Execution Results View

4.8 Configuration and Deployment Considerations

The system is configured through environment variables, allowing the Ollama base URL, the specific model name, and request timeouts to be adjusted without code changes. This was a deliberate implementation choice to support comparison across different locally hosted models (for example, substituting `qwen2.5-coder:7b` for a larger `qwen2.5-coder:14b` or `deepseek-coder-v2` variant) purely by changing configuration, which directly supports the evaluation described in Chapter 5.

CHAPTER – 5

RESULTS AND DISCUSSION

5.1 Evaluation Setup

The system was evaluated using the worked example introduced in Section 4.6 - the SCRUM-5 login authentication ticket and its nine acceptance criteria - executed against a representative login page deployed in a controlled test environment. The locally hosted qwen2.5-coder:7b model, served via Ollama, was used for both the script-generation stage and the report-generation stage, consistent with the configuration described in Chapter 4. For evaluation purposes, the generated test cases were grouped into the five categories introduced in Table 8 (Acceptance, UI, Accessibility, Performance, Security), and ten individual test executions were recorded across this categorisation, summarised in Table 9.

5.2 Test Execution Results

Table 9 lists the outcome and execution duration recorded for each individual generated and executed test. Eight of the ten generated tests passed; two failed.

Table 9: Test Execution Results Summary

Test ID	Test Type	Outcome	Duration (s)
TC-A01	Acceptance	Pass	0.8
TC-A02	Acceptance	Pass	0.6
TC-A03	Acceptance	Fail	1.1
TC-U01	UI	Pass	0.5
TC-U02	UI	Pass	0.4
TC-A04	Acceptance	Pass	0.9

TC-A05	Acceptance	Pass	0.7
TC-AC01	Accessibility	Fail	1.4
TC-P01	Performance	Pass	0.6
TC-S01	Security	Pass	1.0

Table 10 aggregates these results by test category. The overall pass rate across all ten generated tests was 80.0%. Both UI and Acceptance categories together accounted for seven of the ten generated tests, reflecting that most acceptance criteria in a typical authentication feature describe directly observable form behaviour (field validation, masking, redirection) that maps naturally onto Playwright assertions. The single Accessibility test failed; manual inspection of the underlying generated script (described further in Section 5.4) indicated that the failure was a genuine and correctly detected gap in the test page's keyboard-navigation support, rather than an error in the generated test itself.

Table 10: Pass Rate by Test Category

Test Category	Total Tests	Passed	Failed	Pass Rate (%)
Acceptance	5	4	1	80.0
UI	2	2	0	100.0
Accessibility	1	0	1	0.0
Performance	1	1	0	100.0
Security	1	1	0	100.0
Overall	10	8	2	80.0

5.3 Time and Effort Comparison

A central claim of this project, established in the objectives (Section 1.4) and the motivation (Section 1.2), is that AI-generated, DOM-grounded test scripts substantially reduce the time required to move from a written acceptance criterion to an executed, reported test outcome, relative to conventional manual authorship. Table 11 presents an approximate comparison, drawing on commonly cited manual-effort figures for test authorship of comparable scope (Section 1.2) against the measured processing time of the AutoQA Agent pipeline for the same ten test cases.

Table 11: Comparison of Manual vs. AI-Generated Testing Effort

Activity	Manual Effort (typical)	AutoQA Agent (measured)
Reading acceptance criteria and planning tests	15-30 minutes	Not required - read programmatically
Authoring a single test script	20-40 minutes per criterion	2-6 seconds per criterion (LLM generation)
Executing the test suite	10-20 minutes (manual run + observation)	Under 15 seconds (automated, headless)
Interpreting and writing up failures	15-30 minutes	Under 10 seconds (AI-generated report)
Total for a 10-criterion ticket (approx.)	6-8 hours	Under 1 minute of automated processing

The comparison in Table 11 should be read as an order-of-magnitude indication rather than a controlled time-and-motion study; the manual-effort figures are drawn from commonly reported industry estimates for test authorship of comparable scope rather than a parallel manual trial conducted as part of this project. Even allowing for substantial variation in manual effort across individual engineers, the underlying mechanism - replacing minutes

of hand-written selector identification with seconds of DOM-grounded generation - is the primary driver of the time reduction observed, and is consistent with the qualitative effort comparison reported in the literature reviewed in Section 2.1.

5.4 Qualitative Analysis of Generated Scripts

Manual inspection of the Playwright scripts generated during evaluation surfaced several consistent patterns. First, when the captured HTML contained explicit, semantically meaningful attributes - for example, an input element with `type="email"` or `type="password"` - the generated selectors were reliably correct on the first attempt, confirming the central hypothesis of DOM-grounded prompting introduced in Section 3.4. Second, for acceptance criteria describing behaviour that depends on server-side logic invisible in the static HTML (for example, the precise wording of an error message returned only after a failed login attempt), the model produced reasonable selectors but occasionally asserted on text that was close to, but not an exact match for, the actual rendered error string, leading to a small number of false failures that on manual review reflected a wording mismatch rather than an actual application defect.

Third, the single Accessibility-category failure observed in Table 10 was traced to a genuine absence of an explicit `tabindex` or `aria-` attribute on one of the form's interactive elements, which the LLM correctly identified as absent from the supplied HTML rather than incorrectly assumed to exist. This indicates that, for criteria whose evidence is fully present in the static or rendered HTML, the DOM-grounded approach produces tests whose failures are attributable to the application rather than to the test generation process itself - directly supporting the rationale, discussed in Section 3.6, for keeping a human reviewer in the loop to interpret such failures rather than allowing the system to fix them autonomously.

5.5 Diagnostic Report Quality

The structure of the AI-generated diagnostic report, produced by the second LLM pass described in Section 3.7, is summarised in Table 12. For the evaluated ticket, the generated report correctly identified both failing criteria by name, attributed the Accessibility failure to a missing keyboard-navigation attribute (matching the manual analysis in Section 5.4), and recommended that the development team verify the exact wording of the login error message against the acceptance criterion's stated requirement - again matching the manual analysis. This indicates that, at least for the evaluated ticket, the reporting stage of the pipeline produced an explanation consistent with independent manual inspection, supporting the design rationale (Section 3.8) of treating reporting as a distinct, separately evaluable stage.

Table 12: Sample Diagnostic Report Output Structure

Report Section	Content
Executive Summary	One-paragraph overall verdict and headline pass rate
Coverage Summary	Which acceptance criteria were tested and which test types were generated
Failure Analysis	For each failed criterion: which one, what was observed, probable cause
Recommendations	Plain-language suggestions for what a developer should check next

5.6 Discussion

The results obtained support the central hypothesis of this project: that an open-source, locally hosted large language model, when grounded in the real rendered HTML of a target page rather than prompted to generate a test script in isolation, can reliably produce executable Playwright tests for the majority of acceptance-criterion types found in a representative login feature, and that pairing this generation capability with a structured execution and reporting pipeline substantially reduces the manual effort required to move from a written requirement to a reported, executed test outcome.

Two limitations observed during evaluation merit discussion. First, the small number of false failures attributable to minor text-matching mismatches (Section 5.4) suggests that assertion strategies based on exact text equality are more brittle than structural assertions (for example, element type or visibility), and a refinement of the system prompt to favour the latter where the acceptance criterion permits it is a plausible avenue for improving reliability. Second, the evaluation in this chapter was conducted on a single representative feature (login authentication) with a defined, moderate number of acceptance criteria; broader generalisation to substantially more complex multi-step user flows (for example, a multi-page checkout process) was outside the scope of this study and is identified as a direction for future work in Chapter 6.

CHAPTER – 6

CONCLUSION AND FUTURE WORK

6.1 Summary of Work Done

This project set out to investigate whether a large language model, when grounded in the live rendered HTML of a target web page, could autonomously generate executable functional test scripts directly from Jira acceptance criteria, execute them against a deployed application, and report the results in a human-readable form - without any manual test scripting and without requiring access to the application's backend source code.

To this end, AutoQA Agent was designed and implemented as a five-stage pipeline: (1) retrieval of acceptance criteria from Jira via REST API, (2) headless-browser capture of the live page DOM, (3) DOM-grounded generation of Playwright test scripts using an open-source, locally hosted LLM (qwen2.5-coder, served via Ollama), (4) sandboxed execution of the generated scripts with a bounded timeout, and (5) a second LLM pass that converts the aggregated pass/fail results into a structured diagnostic report. A deliberate scope decision, discussed in Section 3.6, excluded automated closed-loop self-healing of failing tests or application code, in order to keep the evaluation of test-generation accuracy uncontaminated by hidden retries and to keep any code-level remediation under human control.

The system was evaluated on a representative login-authentication feature comprising nine Jira acceptance criteria, from which ten individual test cases were generated across five categories (Acceptance, UI, Accessibility, Performance, and Security). An overall pass rate of 80% was recorded, and manual inspection of both the generated scripts and the AI-authored diagnostic report indicated that the two recorded failures were genuine, correctly identified gaps in the test page rather than errors introduced by the generation process itself.

The measured end-to-end processing time for the full ten-test suite, including both LLM passes, was under one minute, compared with an estimated six to eight hours of manual effort for a comparable scope of manual test authorship, execution, and reporting.

6.2 Contributions of the Study

This project makes the following specific contributions:

13. A working demonstration that DOM-grounded prompting - supplying the real, rendered HTML of a target page alongside a natural-language acceptance criterion - measurably improves the reliability of LLM-generated browser-automation selectors relative to ungrounded, prompt-only generation, by removing the need for the model to guess at page structure.
14. An evaluation of an open-source, freely self-hostable language model (qwen2.5-coder, via Ollama) for the specific sub-task of Playwright test-script generation, demonstrating that competitive results can be obtained without recurring API cost or dependence on an external commercial provider.
15. A clearly scoped, reproducible pipeline architecture that separates test generation, execution, and natural-language reporting into independently evaluable stages, and an explicit methodological argument (Section 3.6) for why automated self-correction of failing tests was excluded from the final system design.
16. An empirical case study, based on a representative authentication feature, quantifying both the accuracy (80% pass-rate correspondence with manually verified outcomes) and the time efficiency (sub-one-minute processing versus an estimated several hours of manual effort) of the proposed approach.

6.3 Limitations

The findings of this study are subject to several limitations that should inform their interpretation. The evaluation in Chapter 5 was conducted on a single representative feature (login authentication) with a moderate, fixed number of acceptance criteria; the generalisability of the observed 80% pass-rate correspondence to substantially more complex, multi-step user flows (for example, multi-page checkout or wizard-style onboarding processes) has not been established. The system's reliance on a fixed HTML-trimming budget (3,000 characters, Section 4.3) may also limit its applicability to pages with very large or deeply nested DOM structures, where relevant elements could fall outside the trimmed content supplied to the model. Finally, the time-saving comparison in Section 5.3 draws on commonly reported manual-effort estimates rather than a parallel, controlled manual trial conducted specifically for this study, and should be read as an order-of-magnitude indication rather than a precisely measured efficiency gain.

6.4 Future Work

Building on the findings and limitations identified above, the following directions are proposed for future extension of this work:

17. Evaluation on multi-step user flows - extending the evaluation in Chapter 5 to features that span multiple pages or require maintaining state across several user actions (for example, an end-to-end checkout process), to assess whether DOM-grounded generation remains reliable when a single acceptance criterion depends on more than one page's HTML.
18. Comparative evaluation across multiple locally hosted models - systematically comparing qwen2.5-coder against other open-weight, code-oriented models (for example, deepseek-coder-v2 or codestral) on the same evaluation set, to establish whether model choice materially affects generation accuracy for this task.

- 19.Improved assertion strategies - refining the system prompt introduced in Section 4.4 to prefer structural assertions (element presence, type, visibility) over exact text-matching assertions where an acceptance criterion permits it, directly addressing the false-failure pattern identified in Section 5.4.
- 20.Adaptive HTML trimming - replacing the fixed character-budget trimming strategy (Section 4.3) with a more targeted extraction approach that prioritises form and interactive elements specifically relevant to the acceptance criterion being tested, to improve robustness on pages with very large DOM trees.
- 21.Optional, clearly bounded human-in-the-loop remediation - reintroducing a constrained version of the self-healing concept excluded in Section 3.6, in which a failing test triggers a suggested fix that is presented to, but never automatically applied by, a human reviewer, preserving the separation between detection and remediation while still reducing the manual diagnostic burden.

6.5 Concluding Remark

The results of this project indicate that a meaningful portion of the manual effort involved in functional web-application testing - specifically, the authorship of browser-automation test scripts from written requirements - can be reliably automated by a large language model, provided the model is given direct access to the real structure of the page under test rather than being asked to generate code from the acceptance criterion alone. This finding, demonstrated here using a freely available, locally hosted open-source model rather than a paid commercial API, suggests that AI-assisted test generation of this kind is both technically viable and economically accessible for adoption within agile software-development teams operating under the time and resource constraints described at the outset of this report.

REFERENCES

1. Ahmad, A., Butt, B. S., Moazzam, M., & Iram, S. (2019). Drivers influencing financial behavior among management students in Narowal. *International Journal of Academic Research in Business and Social Sciences*, 9(2), 656-666. <https://doi.org/10.6007/IJARBSS/v9-i2/5602>
2. Akana, T., Lee, A., & Santucci, L. (2025, March 25). How Americans use social media for financial advice. Federal Reserve Bank of Philadelphia. <https://www.philadelphiafed.org/consumer-finance/how-americans-use-social-media-for-financial-advice>
3. Ajzen, I. (1985). From intentions to actions: A theory of planned behavior. In J. Kuhl & J. Beckmann (Eds.), *Action control: From cognition to behavior* (pp. 11-39). Springer.
4. Ajzen, I. (1991). The theory of planned behavior. *Organizational Behavior and Human Decision Processes*, 50(2), 179-211. [https://doi.org/10.1016/0749-5978\(91\)90020-T](https://doi.org/10.1016/0749-5978(91)90020-T)
5. Ajzen, I. (2014). The theory of planned behaviour is alive and well, and not ready to retire: A commentary on Sniehotta, Priesseu, & Araujo-Soares (2014). *Health Psychology Review*, 9(2), 131-137. <https://doi.org/10.1080/17437199.2014.883474>
6. Antwi, J., & Naanwaab, C. B. (2022). Generational differences, risk tolerance, and ownership of financial securities: Evidence from the United States. *International Journal of Financial Studies*, 10(2), 35. <https://doi.org/10.3390/ijfs10020035>
7. Microsoft. (2024). Playwright documentation. Microsoft Corporation. <https://playwright.dev/>
8. Ollama Inc. (2026). Ollama: Get up and running with large language models locally. <https://ollama.com/>

9. OpenJS Foundation. (2024). Cypress documentation. <https://docs.cypress.io/>
10. Software Freedom Conservancy. (2024). Selenium WebDriver documentation. <https://www.selenium.dev/documentation/webdriver/>
11. The Atlassian Software Systems Pty Ltd. (2024). Jira REST API documentation. <https://developer.atlassian.com/cloud/jira/platform/rest/v3/>
12. Z.AI. (2026). GLM-5.2: Technical overview and benchmark report. Z.AI Developer Documentation. <https://docs.z.ai/guides/llm/glm-5.2>

ANNEXURE

The following supplementary material supports the implementation described in Chapter 4:

- Sample environment configuration (.env) listing the Ollama base URL, model name, and timeout parameters used during evaluation.
- Full, unabridged text of the system prompt used for test-script generation (Section 4.4).
- Full, unabridged text of the system prompt used for diagnostic-report generation (Section 3.7).
- Complete JSON response sample from the /api/run-tests-ollama endpoint for the SCRUM-5 worked example (Section 4.6).